radeon-fix

A private, CLI-driven AI code-repair agent for local git repositories, running one Qwen3-14B model on an AMD Radeon GPU with ROCm.

Track 2 submission - AMD AI DevMaster Hackathon 2026 (Radeon hackathon 2026-07) | Submission date: 2026-08-05 | Author: Hao Miao

1. Application Scenarios

radeon-fix is designed for users who need a code-repair agent while keeping source code on their own machine or private network. The model, inference server, and agent all run locally; no code is sent to a hosted API.

- - **Private and enterprise repositories:** teams in finance, R&D, or regulated industries that cannot upload proprietary code to public model endpoints. The vLLM endpoint is bound to loopback (127.0.0.1) and the agent never leaves the local GPU host.
- - **Issue-driven bug fixing:** a developer pastes a bug report or failing-test description and receives an auditable unified diff (patch.diff), a Markdown report, the full trajectory, and AMD GPU metrics - without modifying the original repository.
- - **Scriptable CLI workflows:** usable in SSH sessions, containers, and CI-style pipelines. Exit codes distinguish success, no-patch, and fatal errors so callers can automate triage.
- - **AMD RadeonX Cloud GPU tenants:** users who already rent Radeon instances get an inference-only product with no training cost, no external dependency, and measured performance on gfx1100 hardware.

2. Agent Architecture

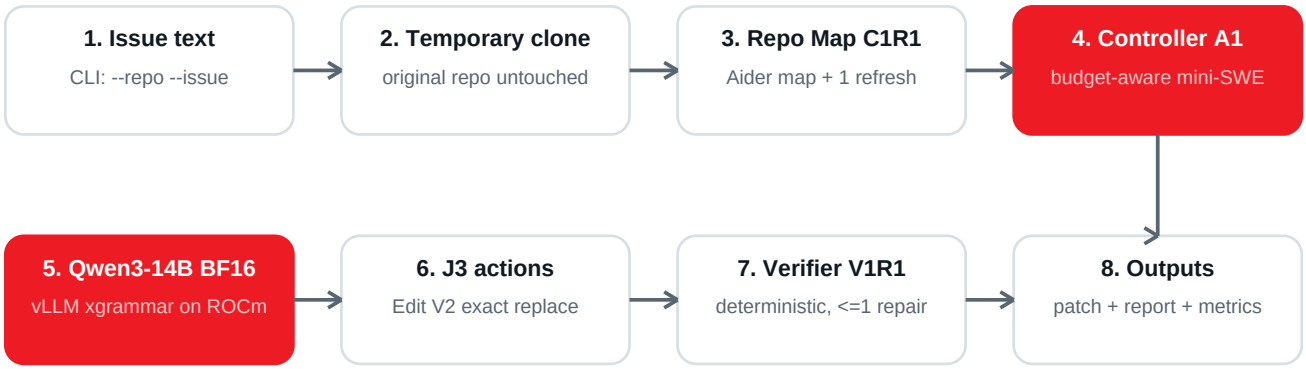

Figure 1. radeon-fix architecture: the CLI clones the repository into a temporary workspace, builds repository context, runs a budget-aware mini-SWE controller against a local vLLM model, and applies a deterministic verifier before emitting artifacts.

The product cell is deliberately narrow and was chosen through controlled ablations: C1R1 (Aider Repo Map with one focused refresh) + J3 (xgrammar structured actions with Edit V2 exact replace) + A1 (budget-aware controller) + V1R1 (deterministic verifier with at most one evidence-guided repair). Qwen Code 0.21.0 is retained as an independent comparator harness; the product itself uses the mini-SWE controller.

3. Core Capabilities

- - **radeon-fix CLI:** `radeon-fix --repo <path> --issue "<text>"` or `--issue-file`, with budgets for model turns (default 12), tool calls (30), and wall time (900 s), plus configurable endpoint, model, seed, and output directory.
- - **Safe workspace:** the repository is cloned into a temporary directory; the original repo is never modified. The patch is reviewed and applied later with `git apply patch.diff`.

- - **Structured action protocol (J3):** inspect, search, replace, edit, execute, and finish actions are emitted as constrained JSON via vLLM xgrammar; Edit V2 uses exact span replacement, so a successful apply is deterministic.
- - **Repository context (C1R1):** an Aider Repo Map (2048 tokens) is provided as initial context, followed by one focused refresh after the first successful inspection.
- - **Budget-aware controller (A1):** a loop guard plus turn/tool/wall-time budgets prevent runaway runs and keep cloud GPU cost predictable.
- - **Deterministic verifier (V1R1):** structural and apply checks after edits, with at most one evidence-guided repair request.
- - **Mainstream CLI UX:** the terminal shows one action line per step (inspect / replace / edit / execute / finish), executed commands, and a compact unified diff (max 24 lines) after each successful edit.

Artifacts written to `./radeon-fix-out/<repo>-<timestamp>/`:

Artifact	Contents
patch.diff	Unified diff of all agent edits, ready for review and git apply.
report.md	Issue, model, trajectory summary, verifier verdict, AMD GPU metrics, apply instructions.
trajectory.json	Full agent trajectory: actions, results, tokens, timings.
run.json	Frozen run configuration and budget.
gpu.json	rocm-smi snapshot when running on AMD Radeon hardware.

4. Model Introduction & Local Deployment Plan

The product uses one local model, **Qwen3-14B** (Qwen/Qwen3-14B) in BF16 precision, served by vLLM 0.25.1 over an OpenAI-compatible endpoint on ROCm 7.2.2 with PyTorch 2.11.0+rocm7.2.2. It is inference-only: no training or fine-tuning is performed, and no external model API is called.

Component	Reference configuration
Hardware	AMD Radeon Graphics (gfx1100), 51.52 GB VRAM, RadeonX Cloud
Model	Qwen3-14B BF16 (local checkpoint), 32K context, single sequence
Inference server	vLLM 0.25.1, OpenAI-compatible /v1 endpoint on 127.0.0.1:8000
Structured output	vLLM structured-outputs backend: xgrammar
KV cache	INT8 per-token-head; GPU memory utilization ~0.90
Prefix caching	Enabled (A/B validated, see Section 5)
Client	radeon-fix CLI; pip install -e .; TRACK2_BASE_URL / TRACK2_API_KEY / TRACK2_MODEL env vars
Launch script	scripts/cloud_start_vllm.sh; refuses to start unless gfx1100 and BF16 support are present

Deployment plan: (1) provision an AMD Radeon GPU instance with ROCm; (2) start the local vLLM server with xgrammar; (3) install the package and set the three environment variables; (4) run radeon-fix against a local git repository. The same stack targets other ROCm Radeon GPUs by changing the device check in the launch script; quantization and context-length options are documented for scaling.

5. Inference-Speed Optimization on AMD Radeon GPU

Four measured optimizations make the local agent fast and predictable on Radeon hardware:

- - **Native BF16 inference on gfx1100:** the 14B model runs in BF16 on ROCm with no CPU offload; VRAM usage was about 46.7 GB of 51.52 GB, leaving headroom for 32K context.

- - **INT8 KV cache:** per-token-head INT8 KV quantization reduces memory pressure and enables long agent trajectories on a single sequence.
- - **vLLM prefix caching:** agent turns repeatedly resend the same prompt prefix. Prefix caching cut time-to-first-token (TTFT) p50 from 3,976 ms to 219 ms at 4K input and from 43,568 ms to 562 ms at 16K input (off vs on), while decode throughput stayed at about 21-24 tokens/s.
- - **xgrammar constrained decoding:** structured actions are generated as valid JSON by construction, eliminating retry latency. In the 20-task confirmation run, 189/189 structured actions were schema-valid; in the five-task development gate, 38/38 were valid with zero JSON retries.

Final service acceptance probe on gfx1100:

Metric	Measured value
TTFT p50 / p95	103.97 ms / 137.38 ms (10-request probe)
Decode throughput p50	25.66 tokens/s
Structured-action schema checks	20/20 valid
Automatic tool calls	10/10 success
Stability requests	30/30 success, no OOM or server crash
Agent-level efficiency (20-task run)	mean 1.92 min/task, P95 6.04 min; 189 tool/action calls; 1,094,595 tokens total

6. Validation Evidence

- - **Development gate (five public SWE-bench Lite tasks, fixed budgets):** the final product cell produced 5/5 non-empty patches, 5/5 applied exact replacements, 38/38 valid J3 actions, and zero integrity, API, or patch-capture failures.
- - **SWE-bench Verified fixed confirmation (seed 0, 20 tasks):** RepoMap Final produced 16/20 non-empty patches versus 1/20 for the original runner, with 189/189 structured actions valid and zero integrity violations. Official Docker evaluation of both frozen patch sets resolved 1/20. This is a small fixed confirmation, not a claim of statistical significance.
- - **Demo project (examples/demo_project, pycalc):** three injected bugs across two modules and seven tests. radeon-fix repaired the repository on the AMD instance in about 2 minutes 20 seconds; the patch applies cleanly and all 7/7 tests pass.
- - **Integrity:** gold patches, test patches, and FAIL_TO_PASS / PASS_TO_PASS fields are never exposed to the agent or context providers; official evaluation runs only after candidate patches are frozen; API keys are read from environment variables only.

7. Submission Contents

This pull request adds the Track2-radeon-fix folder containing: the complete source code repository (radeon_code_agent / radeon-fix CLI, mini-SWE runner, experiment harnesses, tests, configuration), README with environment setup and startup guide, product and deployment documentation, the 3:38 demo video plus replay page and screenshots, this specification, and the supplementary poster.

All materials, project descriptions, and the pull request are written in English.